

Parte 4: Conceptos de Git y Python

Vanesa Fernanda Estrada Cruz

1 Git y GitHub

Planteamiento: Explica la diferencia entre Git y GitHub.

Respuesta: Git es un sistema distribuido de control de versiones. GitHub es una plataforma para alojar repositorios Git y colaborar entre mas programadores, es una red mas abierta.

Explicación: Git permite registrar commits, crear ramas y consultar el historial. GitHub aloja una copia remota y ofrece Pull Requests, revisiones y otras herramientas de colaboración, para hacer mas colaborativa la red de programadores.

2 gitignore

Planteamiento: Explica para qué sirve .gitignore.

Respuesta: Indica patrones de archivos y carpetas que Git no debe seguir cuando todavía no están bajo seguimiento.

Explicación: Evita agregar archivos locales, temporales o sensibles, como .venv/, __pycache__/, *.pyc, .env y .DS_Store. No elimina del historial archivos ya confirmados ni deja de seguirlos automáticamente. En ese caso se requiere retirarlos del índice; si se publicaron secretos, también deben revocarse.

3 Entorno virtual

Planteamiento: Explica por qué .venv no debe almacenarse normalmente en GitHub.

Respuesta: Porque contiene el intérprete y las dependencias del entorno local, que pueden reconstruirse y dependen de la plataforma.

Explicación: Publicarlo aumenta innecesariamente el tamaño del repositorio y puede introducir rutas o binarios incompatibles con otra computadora. Cada persona crea su .venv e instala las dependencias con `python -m pip install -r requirements.txt`. Se versiona la definición de dependencias, no el entorno completo.

4 requirementstxt

Planteamiento: Explica para qué sirve requirements.txt.

Respuesta: Lista las dependencias de Python necesarias para instalar el entorno del proyecto; puede incluir versiones concretas.

Explicación: Después de instalar requests en el entorno virtual, `python -m pip freeze > requirements.txt` registra las dependencias instaladas y sus versiones. Otra persona puede instalarlas con `python -m pip install -r requirements.txt`. Debe generarse desde el entorno correcto para evitar incluir paquetes ajenos al proyecto.

5 Stage Commit y Push

Planteamiento: Explica la diferencia entre Stage, Commit y Push.

Respuesta: Stage selecciona los cambios que se confirmarán; Commit los registra localmente; Push envía los commits al remoto.

Explicación: Stage es el área de preparación o índice, actualizada con git add. Commit crea una versión en el historial con los cambios preparados y un mensaje. Push publica los commits que faltan en el repositorio remoto; no convierte automáticamente archivos modificados en commits.

6 Varios commits antes de Push

Planteamiento: Explica por qué un repositorio puede tener varios commits antes de realizar un push.

Respuesta: Porque los commits se guardan en el repositorio local y no requieren publicarse inmediatamente.

Explicación: Git permite registrar distintos avances, incluso sin internet. Por ejemplo, puedo confirmar la estructura inicial, después el estado del README y luego las funcionalidades. Un solo Push puede enviar esos tres commits pendientes y conservarlos como cambios separados en el historial remoto.